

OA Topic Mastery - HackerRank Practice

Topic-wise HackerRank list for OA practice. Mostly Interview Preparation Kit + Algorithms classics. Practice beats planning. Check off. No fluff.

How to open a problem

```
slug -> https://www.hackerrank.com/challenges/{slug}/problem
Example: pairs -> https://www.hackerrank.com/challenges/pairs/problem
IPK hub: https://www.hackerrank.com/interview/interview-preparation-kit
```

Rules

- No AI until AFTER you submit (or timebox ends).
- Timer: Easy 20-30 min. Medium 35-50. Hard 60-75.
- Before code: name the pattern + write invariant in one line.
- Stuck 20 min: brute force, then optimize. Editorial only after attempt.
- Failed = re-solve in 2-3 days blank. Need help again = does not count.
- Finish topic list before skipping. Sliding window / pairs is priority.

Order

1 Warm-up -> 2 Hashing -> 3 Arrays -> 4 Two Pointers/Window -> 5 Binary Search -> 6 Stack/Queue -> 7 Greedy/Sort -> 8 Strings -> 9 Linked Lists -> 10 Trees -> 11 Graphs -> 12 DP -> 13 Heap

Short on time: finish 1-5 + 6 + 11 + 12. Many company OAs use HackerRank UI - practice here so the platform is not foreign.

One problem flow

1. Classify pattern (2 min)
2. Invariant / what you track (2 min)
3. Dry-run sample input by hand (3 min)
4. Code + edge cases from constraints
5. Submit. Only then read discussions for gaps
6. Log: pattern | trigger | bug type

1. Warm-up (do first, timed)

Spot it

Build platform comfort. Still use the no-AI rule.

Template

Read constraints. Write brute if tiny n . Else $O(n)/O(n \log n)$.

Do these (HackerRank)

- Sock Merchant (E) | sock-merchant
- Counting Valleys (E) | counting-valleys
- Jumping on the Clouds (E) | jumping-on-the-clouds
- Repeated String (E) | repeated-string

More

Interview Preparation Kit -> Warm-up Challenges

Topic done when

All 4 AC in one sitting under 60 min total.

2. Arrays & Hashing

Spot it

Counts, lookups, frequency maps, two-sum style.

Template

```
Use dict/Counter/set.
for x in a:
    if need in seen: ...
    seen[x] = ...
```

Do these (HackerRank)

- Hash Tables: Ransom Note (E) | [ctci-ransom-note](#)
- Two Strings (E) | [two-strings](#)
- Hash Tables: Ice Cream Parlor (M) | [ctci-ice-cream-parlor](#)
- Sherlock and Anagrams (M) | [sherlock-and-anagrams](#)
- Frequency Queries (M) | [frequency-queries](#)
- Count Triplets (M) | [count-triplets-1](#)
- Strings: Making Anagrams (E) | [ctci-making-anagrams](#)

More

IPK -> Dictionaries and Hashmaps

Topic done when

Ice Cream Parlor + Sherlock and Anagrams without peeking.

3. Arrays (manipulation / simulation)

Spot it

In-place ops, bribes, swaps, difference array.

Template

```
Simulate carefully. Prefer O(n). Difference array for range updates.
```

Do these (HackerRank)

- 2D Array - DS (E) | [2d-array](#)
- Arrays: Left Rotation (E) | [ctci-array-left-rotation](#)
- New Year Chaos (M) | [new-year-chaos](#)
- Minimum Swaps 2 (M) | [minimum-swaps-2](#)
- Array Manipulation (H) | [crush](#)

More

IPK -> Arrays

Topic done when

New Year Chaos + Array Manipulation AC (crush = difference array).

4. Two Pointers / Sliding Window (PRIORITY)

Spot it

Contiguous segment, pairs with difference k, min unfairness window.

Template

```
l = 0
for r in range(n):
    add a[r]
    while INVALID:
        remove a[l]; l += 1
    update ans
# or sort then window of size k
```

Do these (HackerRank)

- Subarray Division (E) | [the-birthday-bar](#)

- Picking Numbers (E) | picking-numbers
- Pairs (M) | pairs
- Max Min (M) | angry-children
- Minimum Absolute Difference (E) | minimum-absolute-difference-in-an-array
- Sherlock and Array (E) | sherlock-and-array
- Special String Again (M) | special-palindrome-again
- Beautiful Triplets (E) | beautiful-triplets

More

Search Algorithms domain for 'subarray' / IPK Search + Greedy Max Min

Topic done when

Pairs + Max Min + Subarray Division cold. State expand/shrink rule each time.

5. Binary Search / Search

Spot it

Sorted search, rank queries, min unfairness after sort.

Template

```
lo, hi = ...
while lo < hi:
    mid = (lo+hi)//2
    if check(mid): hi = mid
    else: lo = mid + 1
```

Do these (HackerRank)

- Pairs (M) | pairs
- Ice Cream Parlor (M) | ctc-ice-cream-parlor
- Climbing the Leaderboard (M) | climbing-the-leaderboard
- Hackerland Radio Transmitters (M) | hackerland-radio-transmitters
- Maximum Subarray Sum (H) | maximum-subarray-sum (stretch)

More

Algorithms -> Search

Topic done when

Climbing the Leaderboard with binary search, not $O(n^2)$.

6. Stack / Queue

Spot it

Brackets, queue via stacks, histogram rectangle.

Template

```
st = []
for x in s:
    if match(st, x): st.pop()
    else: st.append(x)
```

Do these (HackerRank)

- Balanced Brackets (M) | balanced-brackets
- Queues: A Tale of Two Stacks (M) | ctc-queue-using-two-stacks
- Equal Stacks (E) | equal-stacks
- Largest Rectangle (M) | largest-rectangle
- Castle on the Grid (M) | castle-on-the-grid
- Poisonous Plants (H) | poisonous-plants (stretch)

More

IPK -> Stacks and Queues

Topic done when

Balanced Brackets + Largest Rectangle attempted; brackets AC cold.

7. Sorting + Greedy

Spot it

Sort then one pass. Local choice optimal.

Template

```
a.sort()  
# one pass / window / two pointers
```

Do these (HackerRank)

- Mark and Toys (E) | mark-and-toys
- Sorting: Bubble Sort (E) | ctci-bubble-sort
- Luck Balance (E) | luck-balance
- Greedy Florist (M) | greedy-florist
- Max Min (M) | angry-children
- Minimum Absolute Difference (E) | minimum-absolute-difference-in-an-array
- Sorting: Comparator (M) | ctci-comparator-sorting
- Merge Sort: Counting Inversions (H) | ctci-merge-sort

More

IPK -> Sorting + Greedy Algorithms

Topic done when

Greedy Florist + Max Min without editorial.

8. Strings

Spot it

Frequency, anagrams, consecutive deletes, validity.

Template

```
Counter / two-pointer on chars / DP for LCS-style.
```

Do these (HackerRank)

- Alternating Characters (E) | alternating-characters
- Making Anagrams (E) | ctci-making-anagrams
- Sherlock and the Valid String (M) | sherlock-and-valid-string
- Special String Again (M) | special-palindrome-again
- Common Child (M) | common-child
- Sherlock and Anagrams (M) | sherlock-and-anagrams
- String Construction (E) | string-construction

More

IPK -> String Manipulation

Topic done when

Valid String + Common Child (LCS) AC.

9. Linked Lists

Spot it

Pointer rewiring. Draw first.

Template

```
prev, cur = None, head  
while cur:  
    nxt = cur.next  
    cur.next = prev  
    prev, cur = cur, nxt
```

Do these (HackerRank)

- Detect a Cycle (E) | ctci-linked-list-cycle
- Insert a node at a specific position (E) | insert-a-node-at-a-specific-position-in-a-linked-list

- Insert into Sorted Doubly Linked List (E) | insert-a-node-into-a-sorted-doubly-linked-list
- Reverse a Doubly Linked List (E) | reverse-a-doubly-linked-list
- Find Merge Point of Two Lists (E) | find-the-merge-point-of-two-joined-linked-lists

More

IPK -> Linked Lists

Topic done when

Cycle + Merge Point cold.

10. Trees

Spot it

Height, BST check, LCA.

Template

```
def dfs(node):
    if not node: return base
    L, R = dfs(node.left), dfs(node.right)
    return combine(node, L, R)
```

Do these (HackerRank)

- Tree: Height of a Binary Tree (E) | tree-height-of-a-binary-tree
- Tree: Huffman Decoding (M) | tree-huffman-decoding
- Is This a Binary Search Tree? (M) | ctc-is-binary-search-tree
- BST: Lowest Common Ancestor (E) | binary-search-tree-lowest-common-ancestor
- Tree: Level Order Traversal (E) | tree-level-order-traversal
- Tree: Top View (E) | tree-top-view

More

IPK -> Trees / Data Structures -> Trees

Topic done when

BST check + LCA without looking.

11. Graphs (BFS / DFS)

Spot it

Shortest unweighted, connected components, grid flood fill.

Template

```
q = deque([s]); dist[s]=0
while q:
    u = q.popleft()
    for v in g[u]:
        if v not in dist:
            dist[v]=dist[u]+1; q.append(v)
```

Do these (HackerRank)

- BFS: Shortest Reach in a Graph (H) | ctc-bfs-shortest-reach
- DFS: Connected Cell in a Grid (H) | ctc-connected-cell-in-a-grid
- Roads and Libraries (M) | torque-and-development
- Journey to the Moon (M) | journey-to-the-moon
- Find the Nearest Clone (M) | find-the-nearest-clone
- Even Tree (M) | even-tree
- Castle on the Grid (M) | castle-on-the-grid

More

IPK -> Graphs

Topic done when

BFS Shortest Reach + Connected Cell AC.

12. Recursion / Backtracking / DP

Spot it

Ways to climb, optimal subsequence, candies distribution.

Template

```
dp[i] = best over previous states  
# or recurse + memo
```

Do these (HackerRank)

- Recursion: Fibonacci Numbers (E) | [ctci-fibonacci-numbers](#)
- Recursion: Davis' Staircase (M) | [ctci-recursive-staircase](#)
- Max Array Sum (M) | [max-array-sum](#)
- Candies (M) | [candies](#)
- Abbreviation (M) | [abbr](#)
- Common Child (M) | [common-child](#)
- The Coin Change Problem (M) | [coin-change](#)
- Equal (M) | [equal](#)

More

IPK -> Recursion and Backtracking + Dynamic Programming

Topic done when

Davis Staircase + Candies + Max Array Sum: write state before code.

13. Heap / Priority Queue

Spot it

Running median, kth, merge by cost.

Template

```
import heapq  
heapq.heappush(h, x)  
heapq.heappop(h)
```

Do these (HackerRank)

- QHEAP1 (E) | [qheap1](#)
- Jesse and Cookies (E) | [jesse-and-cookies](#)
- Find the Running Median (H) | [find-the-running-median](#)
- Minimum Average Waiting Time (H) | [minimum-average-waiting-time](#) (stretch)

More

Data Structures -> Heap

Topic done when

Jesse and Cookies + Running Median attempted.

Weekly volume

- Weekdays: 1-2 timed HR problems from current topic.
 - Weekend: full IPK section timed, or 3 mediums / 2 hours, no AI.
 - After IPK core: Algorithms domain filter by topic for extras.
-

Sources

- HackerRank Interview Preparation Kit (official company OA-style set).
- Algorithms / Data Structures domain classics (Pairs, Max Min, Candies...).
- Same topic order as the Codeforces practice sheet for parallel drilling.

Open HackerRank IPK. Start topic 4 if window/pairs is the hole. Submit.